

ToBigs 25기 MLOps & Database

6주차 과제

실시간 AI 패션 추천 서비스 '투빅스타일' 설계

제출자: 김준영

이메일: jykim1428@hufs.ac.kr

제출일: 2026년 4월 17일

과제 개요

본 과제는 실시간 AI 패션 추천 서비스 '투빅스타일'을 구축하는 상황을 가정하고, 다음 세 가지 관점에서 서비스의 기술 인프라를 설계하는 것을 목적으로 한다.

- 서술형 1번 — 서비스 기능별 최적의 데이터베이스 아키텍처 설계
- 서술형 2번 — MLflow · Docker · Apache Airflow · CT Pipeline을 활용한 TO-BE MLOps 인프라 설계 (문제 A, B 해결)
- 서술형 3번 — 운영팀과 데이터팀을 위한 MLOps 관리자 대시보드 설계

각 답변은 발표자료의 핵심 개념(특히 DB 조합 사례, MLOps 4대 구성요소, 리조트 수요예측 사례)을 근거로 하여 구성하였다.

서술형 1번 — 투빅스타일 DB 아키텍처 설계

1-1. 설계 배경

투빅스타일은 (a) 정형 트랜잭션, (b) 이미지 유사도 검색, (c) 대량 로그 분석, (d) 실시간 응답이 모두 동시에 요구되는 복합 서비스이다. 발표자료 17쪽 '현대 서비스의 데이터 요구사항'에서 지적한 것처럼 단일 DB로는 이 모든 요건을 충족시킬 수 없다. 따라서 23쪽 '현대 서비스의 DB 조합' 사례(쿠팡 = PostgreSQL + Elasticsearch + ClickHouse, 왓챠 = PostgreSQL + ClickHouse + Qdrant)처럼 Polyglot Persistence 전략을 채택하여 각 기능별로 최적의 DB를 조합한다.

1-2. 기능별 DB 매핑

서비스 기능	선택 DB (유형)	선정 이유 (특성 기반)
결제 · 주문 · 회원 관리	PostgreSQL (RDB)	ACID 특성(원자성·일관성·고립성·영속성) 필수. 정형 스키마(주문번호·금액·사용자 ID)와 외래키 제약으로 정합성 보장. Scale-up 방식이지만 트랜잭션 신뢰성이 핵심.
스타일 이미지 유사도 검색	Qdrant (Vector DB)	CLIP 등 멀티모달 임베딩(수백~수천 차원 벡터) 저장 및 ANN(HNSW) 기반 Top-K 코사인 유사도 검색 지원. 밀리초 단위 응답으로 실시간 추천에 필수.
상품 카탈로그 & 검색	Elasticsearch (Search Engine)	역색인(Inverted Index) 구조로 텍스트 키워드·브랜드·태그 검색 최적화. 한글 형태소 분석기(nori) 플러그인 적용 가능. 자동완성·오타 보정 기능 제공.
실시간 추천 캐싱 & 세션	Redis (Key-Value NoSQL)	인메모리 구조로 서브 밀리초 응답. TTL 지원으로 세션·추천 결과 자동 만료. Scale-out 용이. Cache-aside 패턴으로 메인 DB 부하 감소.
사용자 행동 로그 분석	ClickHouse (OLAP / Column)	컬럼 지향 저장 방식으로 수십억 행 집계 쿼리도 초 단위 처리. 클릭 · 체류시간 · 구매전환 등 대용량 로그를 BI 대시보드로 실시간 분석.

1-3. 통합 데이터 흐름

① 사용자 요청 → API Gateway. ② 주문/결제는 PostgreSQL에 기록. ③ '비슷한 스타일 찾기' 버튼 클릭 시 CLIP 임베딩 → Qdrant ANN 검색으로 Top-K 유사 상품 회수. ④ 검색창 입력은 Elasticsearch의 자동완성·필터링 경유. ⑤ 추천 결과는 Redis에 TTL 60초로 캐싱하여 재요청 시

즉시 반환. ⑥ 모든 행동 로그는 Kafka → ClickHouse로 적재되어 야간 배치 분석 및 재학습용 피처 생성에 활용.

1-4. 보조 아키텍처: 데이터 레이크 하우스

운영 DB(OLTP)와 분석 DB(OLAP) 사이에 Lake House(예: Delta Lake + S3)를 두어 원본 로그는 Schema-on-Read 방식으로 장기 보관하고, 정제된 피처만 Feature Store로 승격시킨다. 이는 발표자료에서 강조한 '데이터 레이크 + 데이터 웨어하우스의 장점 결합' 관점과 일치한다.

서술형 2번 — TO-BE MLOps 인프라 설계

2-0. AS-IS 진단

현재 구조는 데이터 과학자가 로컬 Jupyter에서 모델을 학습하고, 결과물(pickle)을 수동으로 서버에 업로드하는 MLOps Level 0에 해당한다. 발표자료에서 지적한 'Hidden Technical Debt'(ML 코드는 시스템의 5% 미만)에 정확히 부합하는 상황으로, ① 환경 불일치에 따른 배포 실패(문제 A), ② 모니터링·재학습 부재에 따른 무성 성능 저하(문제 B)가 필연적으로 발생한다.

2-1. MLOps 구성요소별 역할 매핑

구성요소	역할	문제 A 기여	문제 B 기여
MLflow	실험 추적 · 모델 레지스트리, 파라미터·메트릭·아티팩트 자동 기록.	conda.yaml · requirements.txt 자동 저장 → Docker 빌드 시 동일 환경 재현.	재학습 실험 전건 기록, Staging → Production 승격으로 롤백 용이.
Docker	OS·라이브러리·CUDA까지 이미지로 동결, 환경 격리.	★ 핵심 해결. 개발·운영 환경 동일화로 '내 PC에선 되는데' 문제 근본 차단.	재학습 모델 이미지 v-태그 관리로 안전 배포/롤백 지원.
Apache Airflow	DAG 기반 워크플로우 오케스트레이션(스케줄·의존성·재시도).	CI/CD 파이프라인 오케스트레이터로 도커 빌드 → 테스트 → 배포 자동화.	★ 핵심 해결. 데이터 수집 → 드리프트 감지 → 재학습 → 배포 스케줄 자동화.
CT Pipeline	Prometheus + Grafana + Evidently로 모델 품질 지속 모니터링 및 재학습 트리거.	배포 후 서빙 지표(지연·에러율)를 관측하여 이상 징후 즉시 포착.	★ 핵심 해결. 데이터/컨셉 드리프트 자동 감지 → 재학습 트리거 → 고객 불만 이전에 복구.

2-2. 문제 A 해결 방안 — 환경 재현성 확보

원인 분석

로컬 환경의 Python 3.11 · PyTorch 2.3 · CUDA 12.1 조합이 운영 서버의 Python 3.9 · PyTorch 1.13 환경과 불일치. requirements.txt만으로는 OS 라이브러리·GPU 드라이버까지 포괄할 수 없다.

TO-BE 설계

- (1) **Dockerfile 계층화**: base(CUDA·cuDNN) → deps(pip freeze) → app(모델 코드) 3 단계 멀티스테이지 빌드로 재사용성·빌드 속도 확보.
- (2) **MLflow Autolog**: mlflow.pytorch.autolog() 한 줄로 conda.yaml 자동 생성 → Docker 이미지 빌드 스크립트가 이를 읽어 동일 환경을 동결.
- (3) **이미지 레지스트리 정책**: toobigstyle/recommender:<git-sha>-<model-version> 규칙으로 태깅, Harbor/ECR에 업로드. 운영 서버는 반드시 태그 고정 배포.
- (4) **Kubernetes 배포**: Helm Chart로 선언적 배포, readinessProbe로 모델 로드 완료 확인 후 트래픽 라우팅.

결과: 동일 이미지 = 동일 결과. 개발자 로컬·스테이징·프로덕션 전 구간에서 버전 오류 원천 차단.

2-3. 문제 B 해결 방안 — 성능 저하 자동 감지 & 재학습

원인 분석

패션 트렌드는 계절·유행·연예인 이슈에 따라 급변하므로, 학습 당시 데이터 분포(예: 작년 가을 스타일)가 현재 사용자 행동 분포와 괴리되는 Data Drift와 사용자 선호 변화에 따른 Concept Drift가 발생한다. 모니터링이 없으면 CTR·구매전환이 서서히 하락하다가 고객 민원이 누적된 뒤에야 인지된다.

TO-BE 설계 — Continuous Training 루프

- ① **데이터 수집 Layer**: Kafka → ClickHouse → Feature Store(Feast)로 실시간 피쳐 적재.
- ② **모니터링 Layer (Prometheus + Grafana + Evidently)**: 서빙 지연·에러율, 입력 피쳐 PSI(Population Stability Index), 예측 분포 변화 실시간 추적. PSI > 0.2 이거나 CTR 7일 이동평균 10% 하락 시 Alert 발생.
- ③ **Airflow DAG 'ct_retrain'**: 매일 02:00 스케줄 + Alert 트리거. 데이터 검증(Great Expectations) → 피쳐 엔지니어링 → Docker 컨테이너 학습 Job → MLflow 기록.
- ④ **자동 검증 게이트**: 새 모델이 홀드아웃 셋에서 현재 Production 모델 대비 Recall@20 +0.5%p 이상 개선될 때만 Staging → Production 승격.
- ⑤ **Blue-Green / Canary 배포**: BentoML + Istio로 5% → 25% → 100% 단계적 트래픽 전환, 실패 시 즉시 이전 버전 롤백.

결과: 고객 불만 발생 이전에 지표 이상을 포착하여 수 시간 내에 재학습·재배포가 완료되며, 모든 이력은 MLflow에 남아 사후 분석이 용이하다.

2-4. 통합 TO-BE 아키텍처 요약

Raw Data → Airflow DAG → Feature Store → (Docker) Training → MLflow Registry → CI/CD(GitHub Actions) → K8s Serving(FastAPI + BentoML) → Prometheus/Grafana/Evidently Monitoring → Drift 감지 → Airflow Retrain DAG → 반복.
이는 발표자료의 'MLOps Level 2: CI/CD 자동화' 성숙도에 해당한다.

서술형 3번 — MLOps 관리자 대시보드 설계

3-1. 설계 원칙

- Persona-Driven: 운영팀은 '지금 서비스가 정상인가?', 데이터팀은 '모델이 여전히 똑똑한가?'라는 서로 다른 질문을 가진다. 화면을 분리하되 공통 핵심 지표는 상단 요약바 (Top Bar)에 노출.
- Single Pane of Glass: Grafana를 허브로 두고 Prometheus(인프라·서빙), MLflow(실험), Airflow(파이프라인), Evidently(드리프트) 지표를 데이터 소스로 통합.
- Actionable First: 단순 관측이 아니라 각 알람 카드에 'Rollback / Retrain / Silence' 버튼을 배치하여 원클릭 대응이 가능하도록 한다.

3-2. 대시보드 구성 — 역할별 뷰

운영팀(DevOps/SRE) 뷰	데이터·ML팀 뷰
• API p50 / p95 / p99 지연시간 • 초당 요청 수(QPS), 에러율(5xx %) • GPU/CPU-메모리 사용률, Pod 상태 • 현재 Production 모델 버전 & 배포 시각 • Canary 트래픽 비율 · Blue-Green 상태 • Alert 센터(PagerDuty, Slack 연동) • SLA 충족률 (99.9% 가용성 추적)	• 모델 품질: Recall@20, NDCG, CTR, 구매전환 추세 • Data Drift: Feature PSI 히트맵, KS-통계량 • MLflow Runs 비교 (Parallel Coordinates) • Airflow DAG 성공/실패 히스토리 • A/B 테스트 실험군 vs 대조군 지표 • Feature Store 신선도(Freshness) 지표 • 재학습 예정 스케줄 및 트리거 로그

3-3. 공통 Top Summary (요약 KPI)

상단에는 비즈니스·모델·인프라 핵심 지표 6개를 카드 형태로 노출한다. (발표자료 42쪽 '리조트 수요예측 Admin Example' 형식 참고.)

7일 평균 추천 CTR 14.3% ▲ 0.6%p (전주 대비)	현재 Production 모델 recsys v2.4.1 배포: 04-15 02:13	Drift 지수 (PSI) 0.12 정상 (임계치 0.20)
API p95 지연 142 ms 목표 200ms 이하	금주 예상 구매전환 3.8% 예측구간 3.5 ~ 4.1%	Airflow DAG 성공률 99.2% 최근 7일 125건 중 124 성공

3-4. 화면 와이어프레임 (ASCII)

```

┌────────────────────────── TOP KPI BAR ───────────────────┐ | CTR 14.3% ▲ |
└──────────────────────────┘ |
Prod v2.4.1 | PSI 0.12 ✓ | p95 142ms | SR 99.2% |
┌──────────────────────────┘ | | [Ops View]
┌ [ML / Data View]          | | • Latency / QPS chart | • Recall@20 Trend (7d / 30d) |
├ • Error rate heatmap      | | • Feature PSI Heatmap   | | • Pod / GPU status   | • MLflow
Runs (scatter)              | | • Alert Center □       | | • Airflow DAG timeline | | ↳ [Rollback]
└ [Silence] | ↳ [Trigger Retrain] |
└──────────────────────────┘ |
                                ↓ Drill-down: 클릭
시 MLflow Run · Airflow Task Log로 이동

```

3-5. 알람·액션 플로우

① Prometheus가 PSI 0.22 포착 → ② Grafana Alert가 Slack #mlops-alert 채널로 'Drift 감지' 카드 전송 → ③ 카드의 'Retrain' 버튼 클릭 시 Airflow DAG ct_retrain에 REST API 호출 → ④ 학습 완료 후 MLflow가 신규 모델을 Staging에 자동 등록 → ⑤ Canary 5% 배포, Recall@20 검증 통과 시 100% 승격 → ⑥ 전 과정 타임라인이 대시보드 'Incident Replay' 패널에 기록. 이 흐름으로 운영팀·데이터팀은 각자 관점에서 '문제 → 진단 → 복구'를 하나의 화면에서 수행할 수 있다.

결론

본 과제에서는 투빅스타일 서비스를 대상으로 ① 기능별 특성에 맞춘 Polyglot Persistence DB 아키텍처, ② MLflow · Docker · Airflow · CT Pipeline을 축으로 하는 MLOps Level 2 TO-BE 구조, ③ 운영팀·데이터팀 듀얼 뷰 관리자 대시보드를 제안하였다.

핵심 인사이트는 다음과 같다. 첫째, 단일 DB로는 실시간 추천 서비스의 이질적 요구사항 (트랜잭션·유사도 검색·로그 분석·캐싱)을 충족할 수 없으며, 현대 서비스 사례와 같이 기능별 최적 DB를 조합해야 한다. 둘째, '환경 불일치'와 '성능 저하 무인지' 문제는 각각 Docker 컨테이너화와 CT Pipeline·Airflow 자동화로 구조적으로 해결해야 하며, 개별 도구가 아닌 통합 파이프라인 관점에서 접근해야 한다. 셋째, 대시보드는 관찰 도구를 넘어 '원클릭 복구'가 가능한 액션 허브로 설계될 때 실제 운영 가치를 발휘한다.